

Análise de Código Java

WriteAheadLog — Explicação Técnica

1. Objetivo Geral

A classe **WriteAheadLog** implementa um mecanismo simplificado de *Write-Ahead Logging* (WAL), padrão amplamente utilizado em sistemas de banco de dados para garantir durabilidade e atomicidade das transações. O objetivo é registrar operações de escrita num log temporário antes de aplicá-las efetivamente ao armazenamento principal, permitindo confirmação (*commit*) ou cancelamento (*rollback*) das alterações.

2. Lógica e Algoritmo

O fluxo central divide-se em três etapas: (1) **write()** — acrescenta entradas no formato **"SET chave=valor"** à lista de log sem alterar o store; (2) **commit()** — itera sobre o log, analisa cada entrada via **substring** e **split**, persiste os pares no **HashMap** e sinaliza a transação como confirmada; (3) **rollback()** — descarta o log sem aplicar qualquer alteração, mantendo o store intacto.

3. Conceitos e Recursos Java Utilizados

ArrayList como buffer de log sequencial; **HashMap** como armazenamento chave-valor em memória; flag booleana **committed** para controlo de estado; **IllegalStateException** para rejeitar escritas após *commit*; manipulação de strings com **substring(4)** e **split("=", 2)** para extrair a chave e o valor sem depender de bibliotecas externas.

4. Complexidade

A operação **write()** é $O(1)$ amortizado. O **commit()** é $O(n)$, onde n é o número de entradas no log. O **rollback()** é $O(n)$ para limpar a lista. O **read()** é $O(1)$ em média graças ao **HashMap**. O espaço adicional consumido pelo log é $O(n)$, onde n representa as escritas pendentes da transação corrente.

5. Decisões de Design e Limitações

A implementação é intencionalmente minimalista: não há suporte a múltiplas transações concorrentes nem persistência em disco, o que a torna inadequada para ambientes multi-thread sem sincronização adicional. Além disso, após um *commit*, o objeto torna-se inutilizável (não há método de reset), e a análise manual de strings é frágil se o valor contiver o caractere '=', apesar do limite **split("=", 2)** mitigar parcialmente esse risco.